Git_and_GitHub_Basics_Full_Translation_US_English.md

Git and GitHub: Basic Fundamentals

Course Support Material — Full Translation into US English

Definition

Git and GitHub are fundamental tools for software development. Together, they form a version control and code-hosting system, making it easier to organize projects, maintain a history of changes, and collaborate with a team on the same project.

Git

In simple terms, Git is the version control tool you install on your computer. Its main purpose is to track and record changes made to a project's code in chronological order. With Git, you can:

- **Version your code:** Save “snapshots” of your code so you can return to a specific earlier version (commit), if necessary.
- **Manage teamwork:** Git provides mechanisms for managing changes made by multiple developers to the same file, preventing one person's work from overwriting another's. It helps everyone work with the most up-to-date, integrated version of the project.

GitHub

GitHub is a web platform (and cloud service) that hosts repositories managed by Git. It acts as a social network for developers and projects, allowing you to:

- **Store repositories:** Your projects are saved in the cloud and can be accessed from anywhere.
- **Collaborate and share:** You can choose whether a repository is public or private. On the platform, you can access, contribute to, and download code created by others, as well as interact with the community.
- **Build your portfolio:** The platform lets you add personal and contact information and an activity history, creating a useful portfolio for companies.

Installation and Use

Installation

To use Git, you first need to install it on your computer, taking your operating system into account. For a more detailed tutorial, you can watch the video available on YouTube: https://www.youtube.com/watch?v=NgWExh3bswg.

The installation methods for each operating system are listed below.

Linux

If your operating system is Debian Linux (Ubuntu, Mint, etc.), use the following command to install Git:

```
sudo apt-get install git-all
```

Mac

If your operating system is macOS, there are different ways to install Git. They are described in the “Installing on a Mac” section of the official website: https://git-scm.com/book/pt-br/v2/Come%C3%A7ando-Instalando-o-Git.

Windows

If your operating system is Windows, download the installer and follow its instructions: https://git-scm.com/downloads/win.

Initial Configuration

This section explains how to configure Git and create a repository.

Login / User Identity

To start using Git, the first step is to enter your username and email address. These details are essential because they will be associated with every commit (set of changes) you create and save.

```
git config --global user.name <your name here>
git config --global user.email <your email here>
```

Example:

```
git config --global user.name Fulano
git config --global user.email fulano123@gmail.com
```

Note: Run these commands only once to configure Git's global user, or if you want to change the user associated with this computer.

Creating a Repository

To start version-controlling your code, the project must be configured so Git recognizes it. Navigate to the project folder and run:

```
git init
```

This command creates Git's internal structure, identifying the folder as a repository that will be version-controlled.

Then, while still in the repository folder, check its current state by running:

```
git status
```

This command displays the repository's current state. At this point, you should see output similar to the following:

```
On branch master
No commits yet
nothing to commit (create/copy files and use "git add" to track)
```

This tells us that we are on the main branch (`main`) and that there are no commits yet, which is expected because we have not made any changes.

Note: If you run `git init` in a folder that already contains files, the existing files will appear as “untracked.”

First Commit

Assuming your repository was empty when it was created, all file modifications, creations, and deletions will be tracked. Let's create a file named `test.py` to see how this works.

After creating the file, running `git status` will produce output similar to the following:

```
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        test.py
nothing added to commit but untracked files present (use "git add" to track)
```

This tells us that the file `test.py` has been detected by Git, but it has not yet been prepared to be saved in the project's history. Two commands are needed to commit this file.

1. Preparing Changes (`git add`)

The `git add` command performs staging: it moves the changes you made to the staging area.

To stage only the `test.py` file:

```
git add test.py
```

To stage all changes detected in the repository (new files, modifications, and deletions), use this shortcut:

```
git add .
```

After `git add`, the changes are ready to be committed.

2. Saving Changes (`git commit`)

To save the changes staged earlier, use the `git commit` command:

```
git commit -m "Commit description here"
```

The description is the message recorded in version control, marking what was saved at that point. It is essential for the message to be clear and describe what was done.

This allows you to save changes made to your project and, consequently, return to earlier versions if necessary.

Managing Cloud Repositories (GitHub)

So far, you have created and worked with a local repository (only on your computer). To make your repositories accessible from any device and to contribute to and access repositories from other developers, you need to use platforms such as GitHub.

GitHub is recommended because of its popularity and the size of its community, which includes many developers and projects covering a wide range of purposes, technologies, and applications.

Creating Your Account

The first step to managing your repositories on GitHub is to create an account. The process is simple: register on the website at https://github.com/.

After creating your account, you can personalize it by adding your photo, name, contact information, and more.

Working with a Cloud Repository

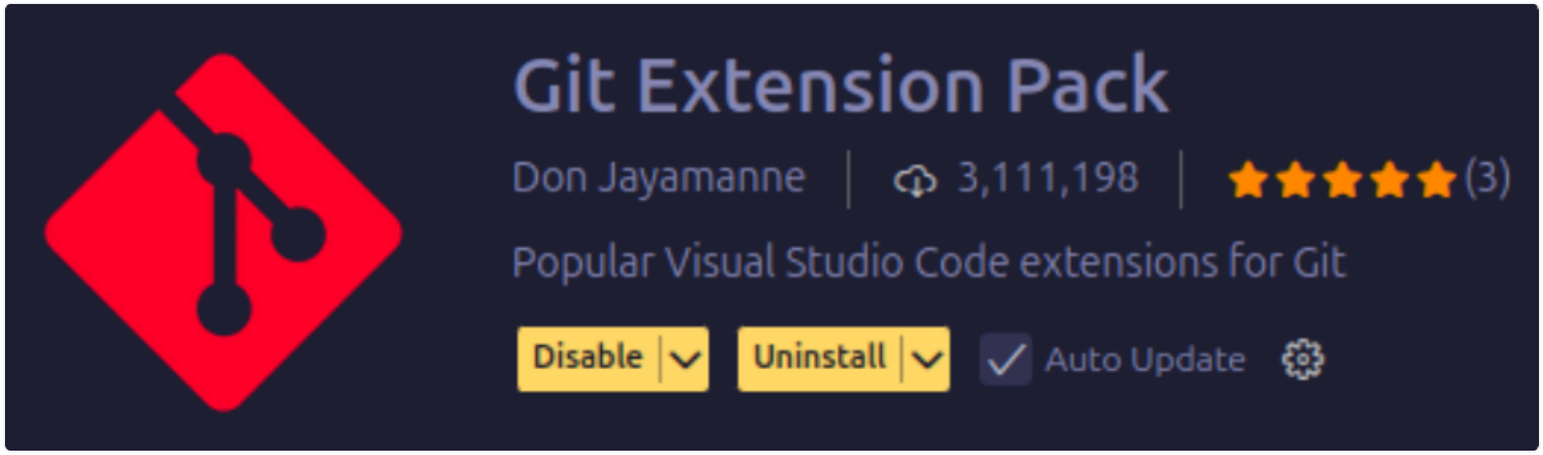
Returning to the repository created locally with Git, you can publish it on GitHub in different ways. For practical purposes, however, we will use the VS Code interface.

If you prefer another way to manage your repository remotely, the following links may help:

- Upload a repository using the command line: command-line tutorial.
- Upload a repository using GitHub Desktop (Windows and Mac): YouTube tutorial.

Preparing VS Code

In VS Code, you need to install the following extension: **Git Extension Pack**. Follow the installation steps shown in the screenshots.



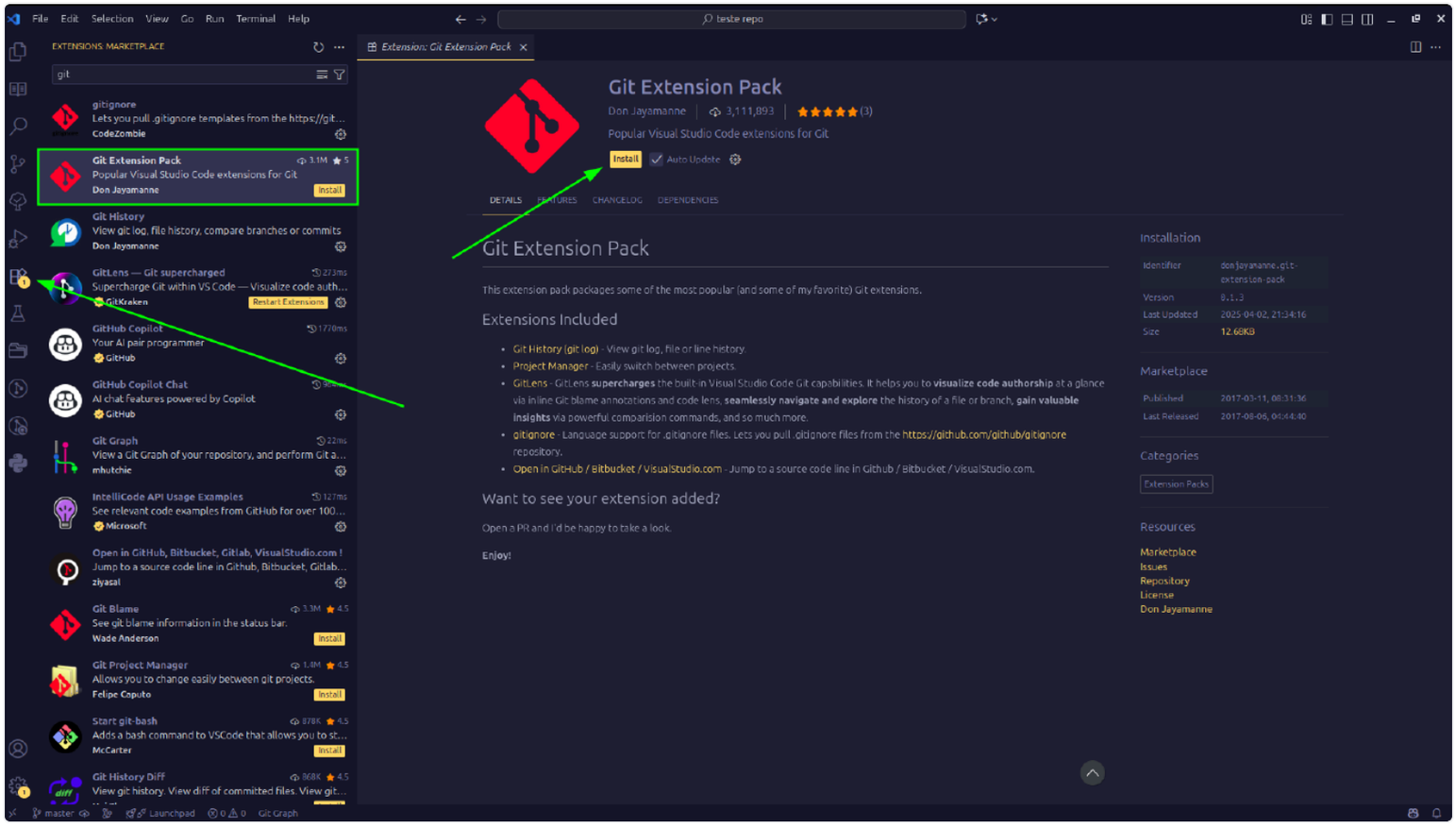
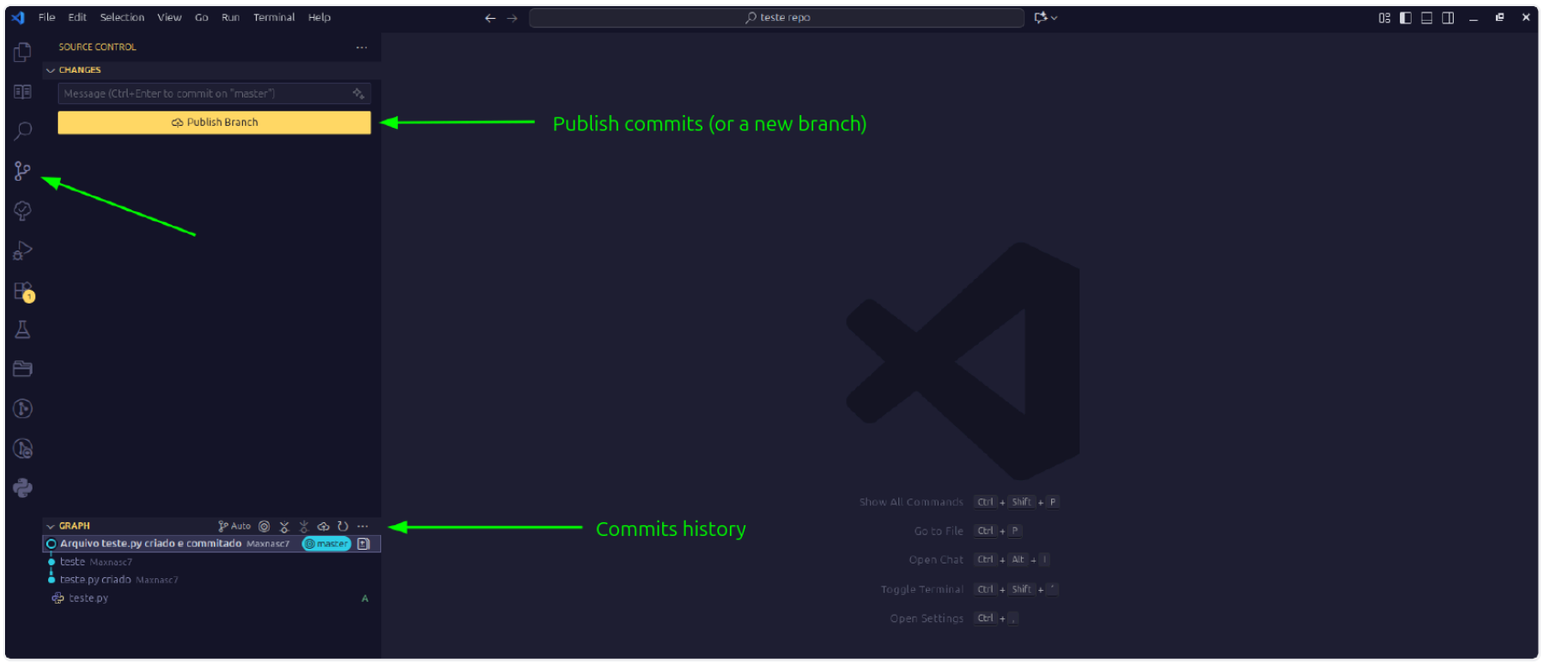


Image in the original material: Git Extension Pack in the VS Code extension marketplace. The extension name is “Git Extension Pack.”

Publishing the Repository

To publish the repository, open the Source Control view (the changes tree) in VS Code. If there are no pending changes (because you have staged everything), the **Publish Branch** option should be available, as shown in the original material.



After clicking the button, you can choose whether the repository will be public and open for everyone to view and download (other people cannot modify your repository unless you allow them), or private. In the latter case, only you have access to the repository, and it does not appear to other people on the platform.

Managing the Remote Repository in VS Code

Once created and published, the repository can be accessed from any computer that has Git installed.

During project development, updating the local repository and updating the remote repository are among the most common tasks, especially when a project is developed by a team. In these cases, the `git pull` and `git push` commands are very useful.

The `git pull` command brings updates from the cloud repository into the local project. To do this, navigate in the terminal to the project folder, or open the VS Code terminal with the project open, and run the command. One example is when you need to bring changes a teammate made to the repository onto your computer.

```
git pull
```

The `git push` command does the opposite: it updates the remote repository with changes you made on your computer. It sends staged and saved commits to the cloud repository (GitHub), allowing another team member —in the example above—to download your changes to the project.

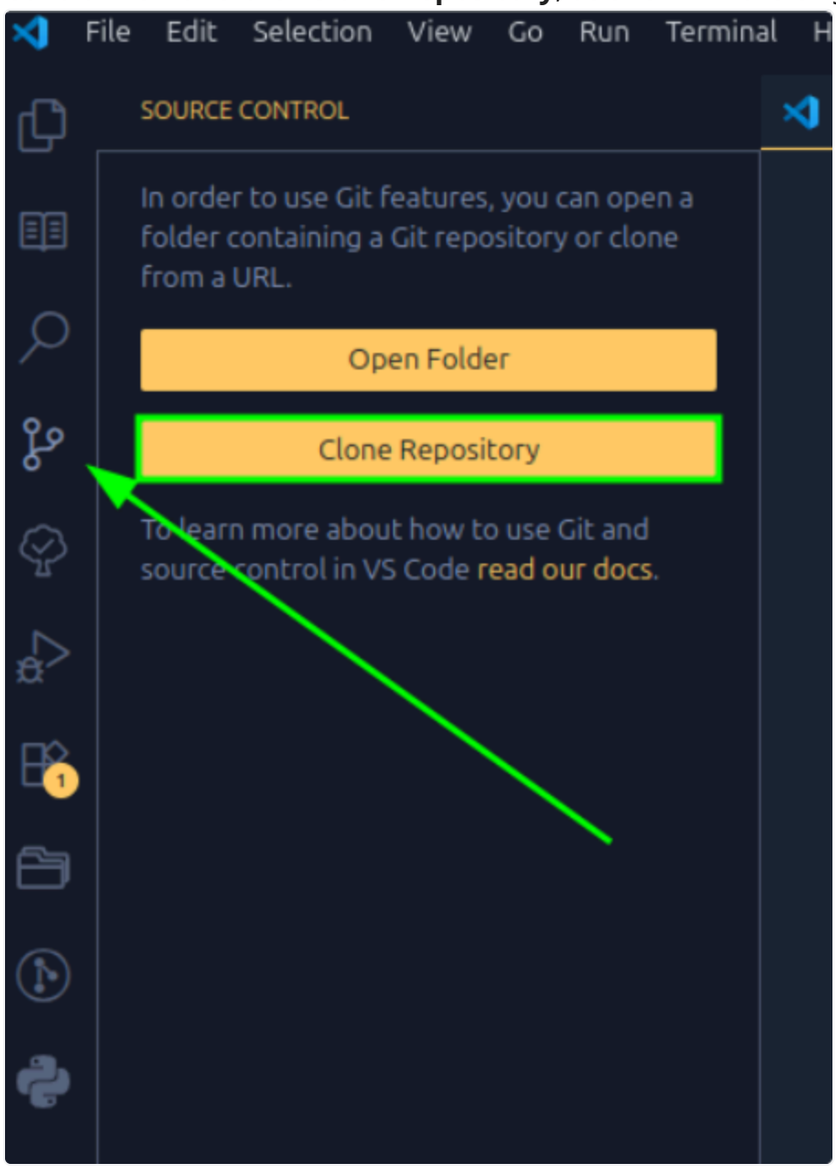
```
git push
```

Accessing Other Repositories

If you need to access a repository that is not already on your computer, whether it belongs to you or someone else, you must clone the repository. Cloning downloads the repository's contents and allows you to edit or update them.

Cloning Repositories Through the VS Code Interface

If you are already signed in, open a new VS Code window (you can use `Ctrl+Shift+N`). Then go to the Source Control view and click **Clone Repository**, as shown in the original screenshot.



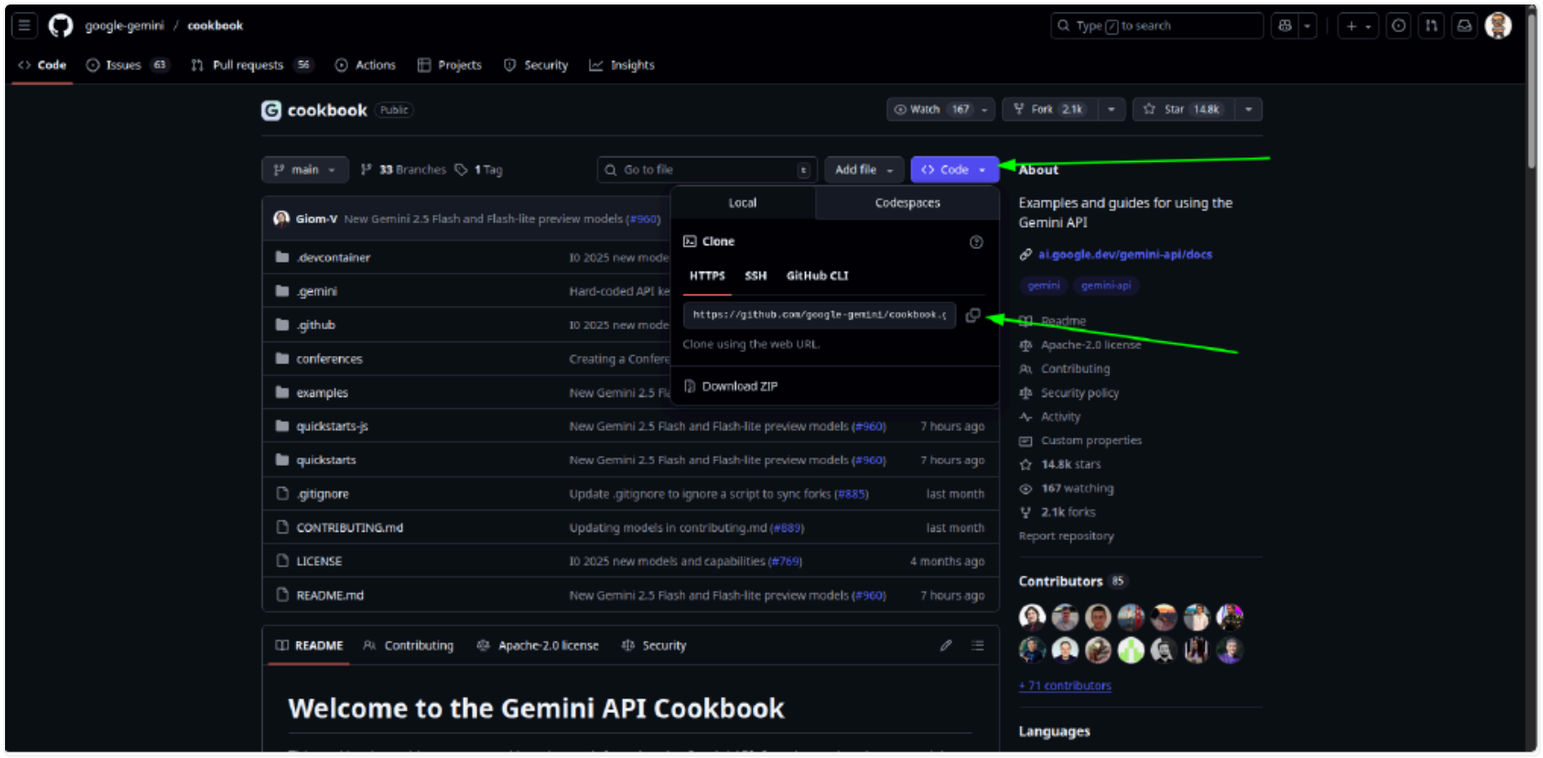
Click **Clone from GitHub**. A list of repositories will appear. Choose the desired repository or type the name of a repository (your own or someone else's), then choose the location where you want to save it.

That's it! After completing these steps, you can use the repository as usual.

Cloning Repositories from the Command Line

You can also clone a repository without using the VS Code interface, by using the command line. Create a folder in the desired location for the repository and run `git clone` followed by the repository link. You can obtain the link from GitHub, as shown in the original example.

```
git clone <repository link>
```



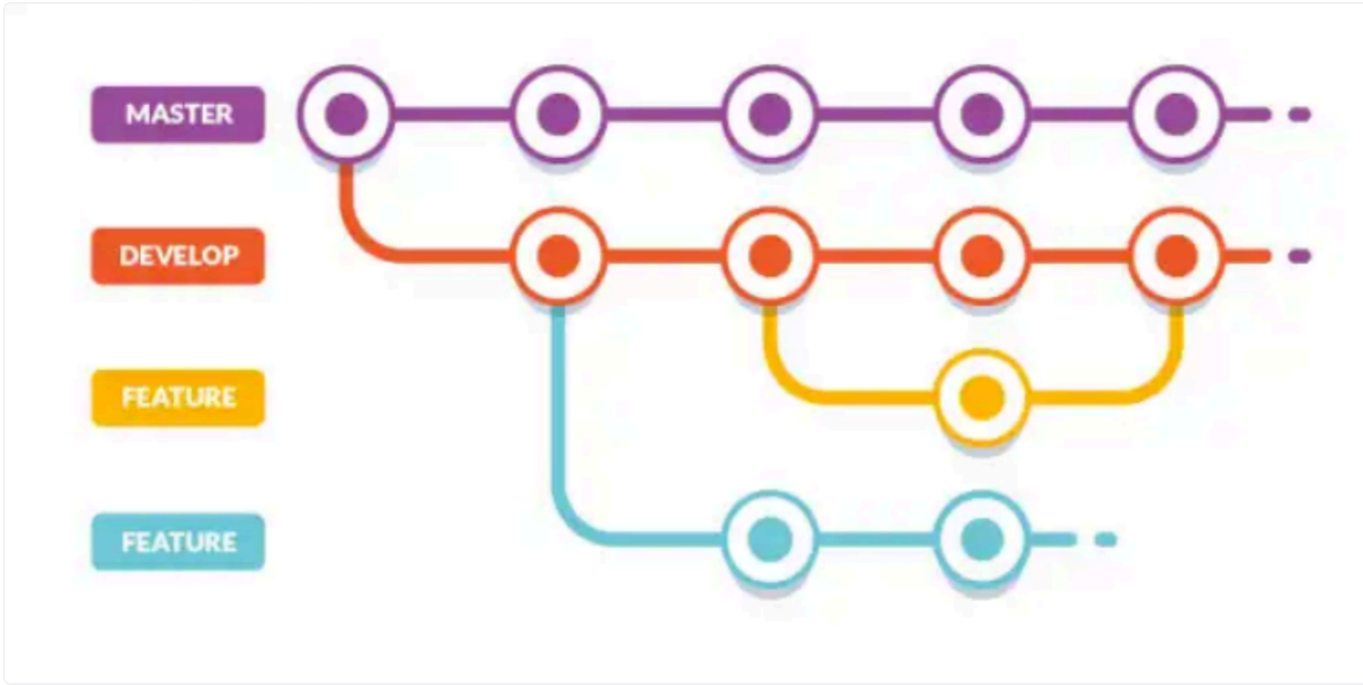
Original screenshot: Illustration of the repository workflow and GitHub link.

Creating Branches

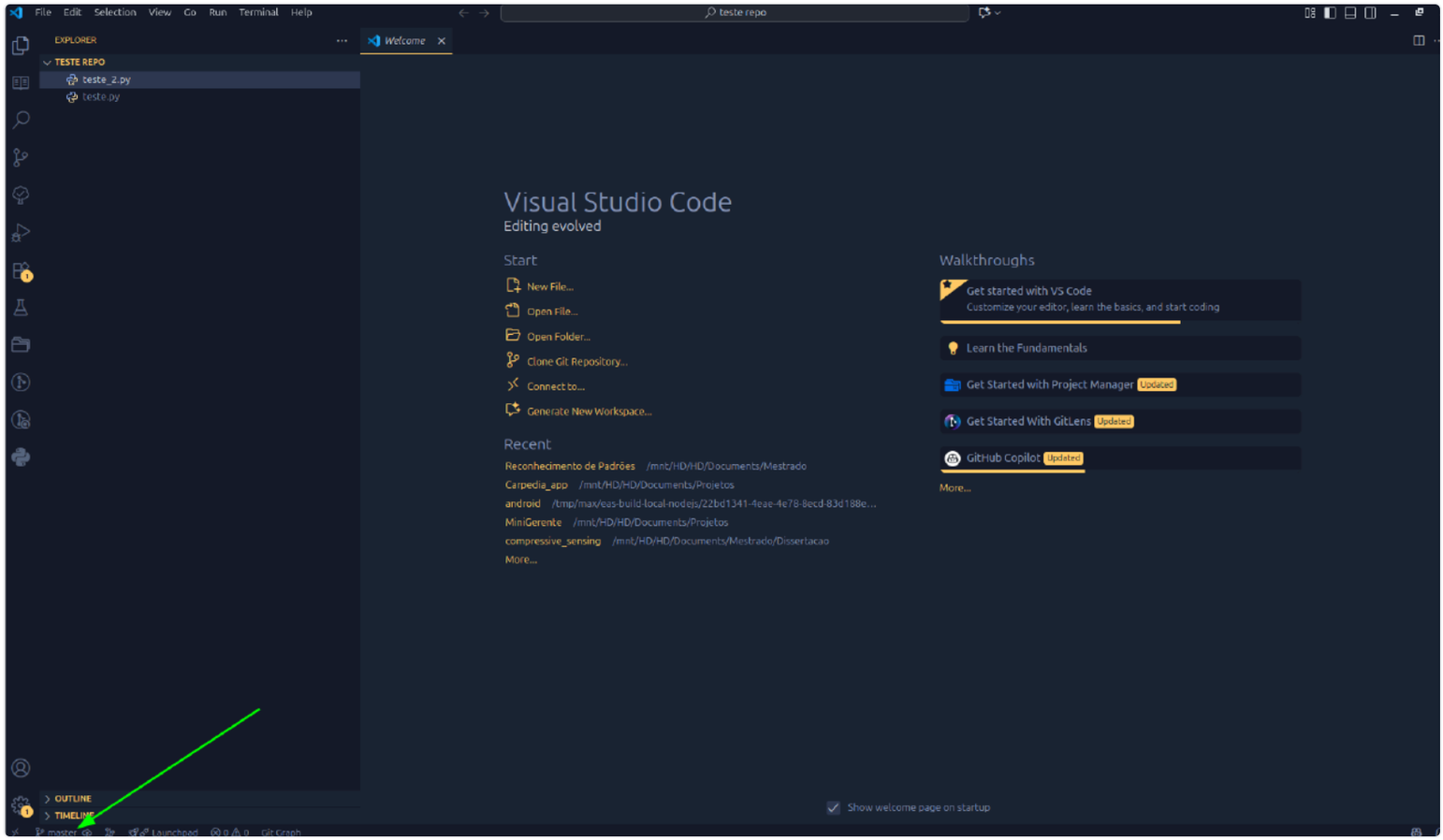
Another project need is creating branches. The purpose of branches is to create a separate line of development in the repository that can receive changes different from those in the main branch (`main` or `master`). A branch can also contain code that is still under development while the `master` branch contains a tested product that is expected to work.

A simplified version of the current branch workflow is as follows. Professional projects have at least three branches: `master` or `main` is the primary, “stable” branch (well established and without errors); `staging` is where code is ready to go to `master` / `main` (also called production), but still needs to be tested; and `develop` is where developers build new features, fix bugs, and update the code.

Usually, several additional branches can be created to implement an isolated feature or to allow each developer to make changes independently.

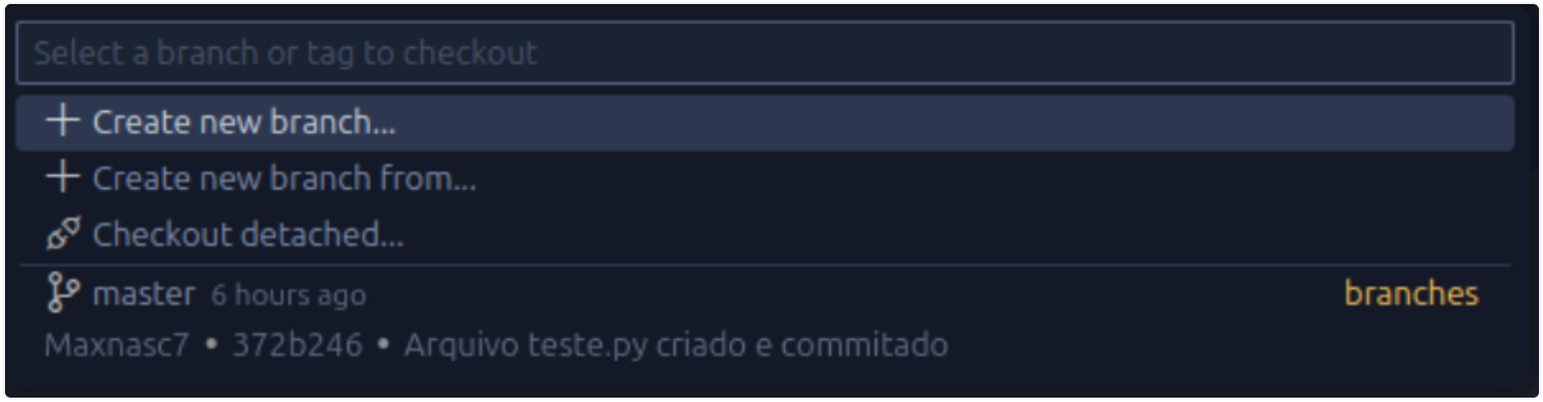


In VS Code, you can manage the current branch and create or delete branches. The branch indicator is located in the lower-left corner of the window.



Managing Branches in VS Code

Click the branch indicator to create a new branch from the current branch (the first option) or create one from a specific branch.



Original screenshot: VS Code branch indicator and branch selection controls. Relevant English interface labels include "Create New Branch" and "Checkout to...".

Next Steps

Git and GitHub are broad topics and extremely useful in the day-to-day work of any developer or company. Improving your knowledge and mastering these tools is essential for entering the job market in the many areas of technology.

For this reason, after the topics discussed in this course, you can explore concepts such as:

- `git merge` and `git rebase` (merge strategies).
- `git reset` and `git revert` (undoing changes).
- `git log` (viewing commit history).
- Pull requests (requests to edit or contribute to other people's repositories).

References

- **Alura:** "What are Git and GitHub?" — https://www.alura.com.br/artigos/o-que-e-git-github?srsltid=AfmBOorzqhEorjQmzeijVOiIz2FXXVWt3Ua7H0l5N4iZ_3LdkubPeaCm
- **Git SCM:** "Getting Started — Installing Git" — https://git-scm.com/book/pt-br/v2/Come%C3%A7ando-Instalando-o-Git

Translation of Text Inside Screenshots

The original screenshots are retained as visual references. Some interface text is embedded in the screenshots and cannot be replaced without recreating the software images. The key Portuguese labels are translated here for reference:

- "Arquivos não monitorados" = "Untracked files."

  "nada adicionado ao envio" = "nothing added to commit."

- “Clonar repositório” = “Clone Repository.”
- “Publicar branch” = “Publish Branch.”
- “Criar nova branch” = “Create New Branch.”

Original Git status example: see the translations above for Portuguese labels embedded in the screenshot. The original material also lists www.alura.com.br as a reference website.